

WEEK-10

Task-1: Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

```
pavani@DESKTOP-380LEIN:~/practical$ echo "Hello Linux" > original.txt
pavani@DESKTOP-380LEIN:~/practical$ ls -li original.txt
42709 original.txt
pavani@DESKTOP-380LEIN:~/practical$ stat original.txt
  File: original.txt
  size: 12          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 42709        Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/   pavani)   Gid: ( 1000/   pavani)
Access: 2026-09-26 12:16:14.837432078 +0000
Modify: 2026-09-26 12:16:14.837432078 +0000
Change: 2026-09-26 12:16:14.837432078 +0000
 Birth: 2026-09-26 12:16:14.837432078 +0000
pavani@DESKTOP-380LEIN:~/practical$ ln original.txt hardlink.txt
pavani@DESKTOP-380LEIN:~/practical$ ls -li
total 232
34757 -rwxr-xr-x 1 pavani pavani 16248 Sep 17 16:36 address
34758 -rw-r--r-- 1 pavani pavani   703 Sep 17 16:34 address.c
13641 -rw-r--r-- 1 pavani pavani    79 Sep 24 14:43 copy.txt
34761 -rwxr-xr-x 1 pavani pavani 16288 Sep 17 17:09 cow
42710 -rw-r--r-- 1 pavani pavani   1015 Sep 17 17:09 cow.c
42709 -rw-r--r-- 2 pavani pavani    12 Sep 26 12:16 hardlink.txt
13628 -rw-r--r-- 1 pavani pavani    36 Sep 26 12:12 input.txt
13625 -rwxr-xr-x 1 pavani pavani 16224 Sep 24 14:32 lowlevel_copy
13631 -rw-r--r-- 1 pavani pavani   679 Sep 21 13:07 lowlevel_copy.c
34759 -rwxr-xr-x 1 pavani pavani 16080 Sep 17 16:40 maps
34760 -rw-r--r-- 1 pavani pavani   266 Sep 17 16:40 maps.c
13614 -rwxr-xr-x 1 pavani pavani 18104 Sep 17 17:05 memory
34762 -rw-r--r-- 1 pavani pavani   1080 Sep 17 16:52 memory.c
13624 -rwxr-xr-x 1 pavani pavani 16432 Sep 26 12:12 mmap_file
13672 -rw-r--r-- 1 pavani pavani   951 Sep 26 12:12 mmap_file.c
42709 -rw-r--r-- 2 pavani pavani    12 Sep 26 12:16 original.txt
13670 -rw-r--r-- 1 pavani pavani    36 Sep 26 12:14 output.txt
13668 -rwxr-xr-x 1 pavani pavani 16128 Sep 24 15:53 redirect
13669 -rw-r--r-- 1 pavani pavani   751 Sep 24 15:43 redirect.c
13663 -rwxr-xr-x 1 pavani pavani 16216 Sep 24 15:15 seek
13664 -rw-r--r-- 1 pavani pavani   374 Sep 21 13:15 seek.c
13665 -rwxr-xr-x 1 pavani pavani 16216 Sep 24 14:41 std_copy
13666 -rw-r--r-- 1 pavani pavani   607 Sep 21 13:25 std_copy.c
13667 -rw-r--r-- 1 pavani pavani    79 Sep 24 14:46 std_copy.txt
13673 -rwxr-xr-x 1 pavani pavani 16216 Sep 26 12:14 traditional
13674 -rw-r--r-- 1 pavani pavani   609 Sep 26 12:13 traditional.c
pavani@DESKTOP-380LEIN:~/practical$ echo "Added through hard link" >> hardlink.txt
pavani@DESKTOP-380LEIN:~/practical$ cat original.txt
Hello Linux
Added through hard link
```

Report: The inode structure of Linux files was investigated using `ls -li`, `stat`, and `find`. A hard link was created and both filenames were found to share the same inode.

A symbolic link was also created and was found to have a separate inode containing a reference to the original path.

The experiment demonstrated how hard and symbolic links affect inode allocation and file access.

Task-2: Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and Implementation complexity.

```
#include <string.h>

int main() {
    int fd;
    struct stat st;
    char *data;

    fd = open("input.txt", O_RDWR);

    if (fd == -1) {
        perror("open");
        return 1;
    }

    if (fstat(fd, &st) == -1) {
        perror("fstat");
        close(fd);
        return 1;
    }

    data = mmap(NULL, st.st_size,
                PROT_READ | PROT_WRITE,
                MAP_SHARED, fd, 0);

    if (data == MAP_FAILED) {
        perror("mmap");
        close(fd);
        return 1;
    }

    printf("Original content:\n");
    write(STDOUT_FILENO, data, st.st_size);

    printf("\n");

    /* Modify file through mapped memory */
    if (st.st_size >= 5)
        memcpy(data, "HELLO", 5);

    msync(data, st.st_size, MS_SYNC);

    printf("File modified using mmap().\n");

    munmap(data, st.st_size);
    close(fd);

    return 0;
}
```

OUTPUT:

```
pavani@DESKTOP-380LEIN:~$ cd practical
pavani@DESKTOP-380LEIN:~/practical$ echo "Hello from Linux memory mapped I/O!" > input.txt
pavani@DESKTOP-380LEIN:~/practical$ nano mmap_file.c
pavani@DESKTOP-380LEIN:~/practical$ gcc mmap_file.c -o mmap_file
pavani@DESKTOP-380LEIN:~/practical$ ./mmap_file
Original content:
Hello from Linux memory mapped I/O!

File modified using mmap().
pavani@DESKTOP-380LEIN:~/practical$ cat input.txt
HELLO from Linux memory mapped I/O!
```

```
GNU nano 8.7.1 traditional.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd, out;
    char buffer[1024];
    ssize_t n;

    fd = open("input.txt", O_RDONLY);

    if (fd == -1) {
        perror("open");
        return 1;
    }

    out = open("output.txt",
               O_WRONLY | O_CREAT | O_TRUNC,
               0644);

    if (out == -1) {
        perror("open");
        close(fd);
        return 1;
    }

    while ((n = read(fd, buffer, sizeof(buffer))) > 0)
        write(out, buffer, n);

    close(fd);
    close(out);

    printf("File copied using read()/write().\n");

    return 0;
}
```

OUTPUT:

```
pavani@DESKTOP-380LEIN:~/practical$ nano traditional.c
pavani@DESKTOP-380LEIN:~/practical$ gcc traditional.c -o traditional
pavani@DESKTOP-380LEIN:~/practical$ ./traditional
File copied using read()/write().
pavani@DESKTOP-380LEIN:~/practical$ cat output.txt
HELLO from Linux memory mapped I/O!
pavani@DESKTOP-380LEIN:~/practical$ _
```

Report:

The program was successfully implemented using `mmap()` for file reading and writing.

The file was mapped into the process's virtual memory and modified directly through the mapped region.

A traditional `read()/write()` implementation was also developed for comparison.

`mmap()` provides convenient direct memory-style access, while `read()/write()` is simpler to implement.

The efficiency of both methods depends on factors such as file size, caching, and access pattern.